

DBMS Exam Notes: Cascading vs. Cascadeless

Core Transaction Management Concepts • GATE / University Exam Rapid Review

1. CASCADING SCHEDULE & SYSTEM PERFORMANCE

A **Cascading Schedule** occurs when one transaction reads data written by another uncommitted transaction. If the initial writing transaction fails and rolls back, all dependent transactions that read its dirty data must also roll back in a chain reaction.

❌ Cascading Schedule Example

Initial State: A = 100

```
T1: READ(A) T1: WRITE(A = 50)
[Uncommitted] T2: READ(A = 50) [Reads
Dirty Data] T3: READ(A = 50) [Reads
Dirty Data] T4: READ(A = 50) [Reads
Dirty Data] T1: FAIL → ROLLBACK (A =
100) -----
RESULT: T2, T3, T4 MUST ALL ROLLBACK!
```

🔥 Why is Cascading a Critical Problem?

- **Wasted CPU Cycles:** CPU spends computation time executing T2, T3, and T4.
- **Performance Degradation:** Rollback operations overhead consumes massive I/O resources.
- **Cascading Chain:** A single transaction crash forces dozens of concurrent operations to undo completed work.

CPU Executes T2, T3, T4



T1 Fails



Chain Rollback (T1, T2, T3, T4)



CPU Work WASTED

2. DIRTY READ CONNECTION

The root cause of cascading rollbacks is the **Dirty Read Phenomenon**: reading uncommitted data modified by another transaction.

★ Core Rule to Remember

Uncommitted WRITE → Subsequent READ → Vulnerable to Dirty Read & Cascading Rollback.

3. CASCADELESS SCHEDULE

A **Cascadeless Schedule** completely eliminates cascading rollbacks by enforcing a strict constraint on read operations.

📌 The Cascadeless Rule

A transaction is **NOT allowed to READ** data written by another transaction until that writing transaction has explicitly **COMMITTED** or **ABORTED**.

✅ Example of a Cascadeless Schedule

```
Initial State: A = 100 T1: READ(A) T1: WRITE(A = 50) T2: READ(A) --> ❌ BLOCKED /
WAITING T1: COMMIT --> [Data A = 50 is now finalized in DB] T2: READ(A) --> ✅ ALLOWED
(Reads safe value: 50)
```

4. FEATURE COMPARISON & QUICK IDENTIFICATION

Feature	Cascading Schedule	Cascadeless Schedule
Read Uncommitted Data	Allowed	Forbidden
Dirty Read	Possible	Prevented
Cascading Rollback	Possible	Prevented
CPU Efficiency	Degraded (Wasted cycles)	High (No chain undo)
Execution Rule	Can read before writer commits	Read only after writer commits/aborts
Remaining Limitation	Chain rollbacks	Write-Write conflicts still possible

★ **Rapid Exam Identification Trick:**

- $WRITE(A) \rightarrow READ(A)$ before COMMIT = **Cascading Schedule**
- $WRITE(A) \rightarrow COMMIT \rightarrow READ(A)$ = **Cascadeless Schedule**

5. LIMITATIONS: THE WRITE-WRITE / LOST UPDATE PROBLEM

Important Exam Distinction: A Cascadeless schedule prevents *Write* $\rightarrow$ *Read* issues, but it **does NOT prevent** *Write* $\rightarrow$ *Write* conflicts.

Write-Write Problem Scenario (Lost Update)

```
Initial State: A = 100 T1: READ(A) -> 100 | T2: READ(A) -> 100 T1: A = A - 10 |
T2: A = A - 20 T1: WRITE(A = 90) | T2: WRITE(A = 80) [Overwrites T1's write!]
```

Since Cascadeless only restricts *reading* uncommitted writes, T2 is allowed to *write* over T1's uncommitted write, leading to a **Lost Update**. This necessitates **Strict Schedules** (Read + Write allowed only after Commit/Abort).

6. REAL-WORLD ANALOGY & EXAM PRACTICE

Student Copying Analogy

Cascading: Student 1 writes an answer. Students 2, 3, and 4 copy it immediately. Student 1 realizes it was wrong and erases it. All 4 students must erase and redo work.

Cascadeless: Student 1 writes an answer and finalizes/submits it (COMMIT). Only then do Students 2, 3, and 4 copy it. No redoing required!

Exam Traps & Flash Solutions

Q1: T1: W(A); T2: R(A); T1: FAIL; $\rightarrow$ **Cascading Rollback / Dirty Read**

Q2: T1: W(A); T1: COMMIT; T2: R(A); $\rightarrow$ **Cascadeless Schedule**

Q3: T1: R(A); T2: R(A); T1: W(A); T2: W(A); $\rightarrow$ **Write-Write / Lost Update Problem**

Final Memory Hierarchy Summary